

「近邻小铺子」项目需求说明书 V1.0

一、项目概述

1.1 项目名称

近邻小铺子

1.2 项目定位

“近邻小铺子”是一个面向固定社区和周边商家的本地购物与配送平台。

平台通过微信小程序为社区居民提供附近店铺浏览、商品购买、微信支付、收货地址管理和配送进度查询等功能。

平台管理员通过后台管理系统维护：

- 店铺；
- 商品；
- 商品分类；
- 配送小区；
- 配送时间；
- 用户；
- 订单；
- 支付和退款；
- 平台经营数据。

1.3 第一版业务模式

第一版采用平台统一运营模式：

- 平台统一创建和管理店铺；
- 平台统一维护店铺商品；
- 平台预先配置可配送小区；
- 每个店铺可以关联多个配送小区；
- 每个店铺可以配置起送金额和配送费；
- 用户一次只能购买一家店铺的商品；
- 一个订单只属于一家店铺；
- 用户通过微信支付完成付款；
- 店铺自行准备商品并负责配送；
- 平台管理员通过后台修改订单状态；
- 第一版暂不开放商家自主入驻。

1.4 第一版建设目标

第一版重点完成以下业务闭环：

选择小区

- 浏览店铺
- 浏览商品
- 加入购物车
- 填写收货地址
- 提交订单
- 微信支付
- 后台接单
- 商品准备
- 开始配送
- 用户查看配送状态
- 订单完成

二、技术架构

2.1 小程序端

采用以下技术：

- uni-app;
- Vue 3;
- TypeScript;
- Pinia;
- Vant Weapp;
- Sass;
- uni.request;
- 微信小程序登录;
- 微信小程序支付;
- 微信小程序订阅消息。

Vant使用说明

小程序端使用的是：

Vant Weapp

不是用于普通Web页面的Vant Vue。

主要使用的组件包括：

- Button;
- Cell;
- Field;
- Popup;
- Dialog;
- Toast;

- Stepper;
- SubmitBar;
- Card;
- Tag;
- Tabs;
- Empty;
- Loading;
- SwipeCell;
- Steps;
- ActionSheet;
- NoticeBar;
- Skeleton。

2.2 服务端

采用以下技术：

- Node.js;
- Koa;
- TypeScript;
- koa-router或 `@koa/router` ;
- MySQL 8;
- Redis;
- Prisma ORM;
- JWT;
- bcrypt;
- Joi或Zod参数校验;
- koa-body;
- koa-static;
- Swagger/OpenAPI;
- node-cron;
- Docker。

服务端主要职责

Koa服务端负责：

- 微信登录;
- 后台登录;
- 用户鉴权;
- 店铺和商品接口;
- 购物车接口;
- 地址接口;
- 订单创建;
- 订单状态流转;
- 微信支付;
- 微信退款;
- 库存处理;
- 定时关闭未支付订单;
- 经营数据统计;
- 操作日志记录。

2.3 后台管理端

采用以下技术：

- Vue 3;
- TypeScript;
- Vite;
- Vue Router;
- Pinia;
- Element Plus;
- Axios;
- ECharts;
- Sass。

Element Plus主要组件

- ElTable;
- ElForm;
- ElDialog;
- ElDrawer;
- ElSelect;
- ElInput;
- ElDatePicker;
- ElTimePicker;
- ElUpload;
- ElPagination;
- ElTag;
- ElTabs;
- ElDescriptions;
- ElStatistic;
- ElMessage;
- ElMessageBox。

2.4 数据和基础设施

- MySQL：保存用户、店铺、商品和订单等业务数据；
- Redis：缓存、分布式锁、登录辅助、订单幂等和限流；
- 对象存储：保存店铺图片、商品图片和轮播图；
- Nginx：反向代理和HTTPS；
- Docker：部署MySQL、Redis、Koa服务和后台管理端；
- 微信支付API：支付和退款；
- 微信订阅消息：订单状态通知。

三、项目角色

3.1 普通用户

普通用户通过微信小程序使用平台。

主要权限：

- 微信登录；
- 选择所在小区；
- 浏览店铺；
- 搜索店铺；
- 浏览商品；
- 加入购物车；
- 管理收货地址；
- 创建订单；
- 微信支付；
- 查看订单；
- 取消符合条件的订单；
- 查看配送进度；
- 申请退款；
- 再次购买。

3.2 平台管理员

平台管理员通过后台管理系统管理整个平台。

主要权限：

- 查看数据看板；
- 管理所有店铺；
- 管理所有商品；
- 管理配送小区；
- 配置店铺配送范围；
- 配置配送时间；
- 查看全部用户；
- 查看全部订单；
- 修改订单状态；
- 处理取消和退款；
- 查看操作日志；
- 管理后台管理员。

3.3 店铺管理员

店铺管理员作为第二阶段功能。

店铺管理员只能管理被授权店铺的数据：

- 查看本店商品；
- 管理本店库存；
- 修改本店营业状态；
- 查看本店订单；
- 接单；
- 修改本店订单状态；
- 查看本店销售统计。

第一版只实现平台管理员，数据库和权限结构预留店铺管理员能力。

四、第一版业务边界

4.1 配送小区

第一版不使用地图自动定位，也不计算用户和店铺之间的真实距离。

平台在后台预先创建固定小区，例如：

- 幸福花园；
- 阳光小区；
- 金色家园；
- 城市之光；
- 万科社区；
- 绿地新城。

用户只能从平台配置的小区列表中选择。

4.2 多店铺规则

平台支持多个店铺，但需要遵循以下规则：

- 一个购物车只能包含一家店铺的商品；
- 一个订单只能属于一家店铺；
- 不同店铺的商品不能合并结算；
- 用户切换店铺添加商品时，需要清空原店铺购物车。

提示示例：

购物车中已有其他店铺商品，是否清空后添加当前商品？

4.3 商品规格

第一版以单规格商品为主。

第一版支持：

- 商品名称；
- 商品图片；
- 商品价格；
- 原价；
- 库存；
- 销量；
- 商品描述；
- 商品备注；
- 购买数量。

第一版暂不支持：

- 多规格SKU；
- 不同规格独立库存；
- 复杂加料组合；
- 规格价格联动。

数据库可以预留SKU扩展能力，但不作为第一版开发范围。

4.4 配送方式

第一版只支持：

商家配送

后续可以扩展：

- 到店自取；
- 平台配送；
- 第三方配送；
- 配送员接单。

4.5 配送进度

第一版采用订单状态节点展示，不做地图实时定位。

配送进度示例：

已支付
→ 商家已接单
→ 商品准备中
→ 待配送
→ 配送中
→ 已送达

4.6 促销活动

第一版暂不支持：

- 优惠券；
- 满减；
- 秒杀；
- 拼团；
- 会员价格；
- 积分抵扣；
- 多店铺合并优惠。

数据库金额字段可以预留优惠金额。

五、小程序端功能需求

5.1 微信登录

功能说明

用户进入小程序时，系统调用微信登录能力获取临时登录凭证，再由Koa服务端换取OpenID。

登录流程

```
用户打开小程序
→ 调用uni.login
→ 获得微信code
→ code发送到Koa服务端
→ 服务端调用微信接口
→ 获得OpenID
→ 创建或查询用户
→ 生成JWT
→ 小程序保存Token
```

用户信息

系统保存：

- 用户ID；
- OpenID；
- UnionID；
- 昵称；
- 头像；
- 手机号；
- 当前选择的小区；
- 用户状态；
- 注册时间；
- 最近登录时间。

登录策略

浏览店铺和商品可以允许未完整授权用户访问。

以下操作必须登录：

- 添加服务端购物车；
- 管理地址；
- 提交订单；
- 支付；
- 查看个人订单；
- 申请退款。

用户首次下单时必须绑定手机号。

5.2 小区选择

页面功能

- 显示平台启用的小区；
- 显示当前选择的小区；
- 支持手动切换小区；
- 保存最近一次选择；
- 根据小区筛选店铺。

首次进入规则

用户首次进入小程序时，可以先浏览全部营业店铺。

用户准备结算时，如果未选择小区，则必须先选择小区。

小区切换规则

用户切换小区后：

- 首页重新查询可配送店铺；
 - 当前购物车暂时保留；
 - 进入结算时重新校验当前店铺是否支持新小区；
 - 如果不支持，则禁止下单。
-

5.3 首页

页面结构

顶部导航
当前小区
店铺搜索框
平台公告
店铺列表
底部TabBar

首页内容

第一版展示：

- 当前小区；
- 搜索框；
- 平台公告；
- 店铺列表；
- 店铺营业状态；

- 空状态；
- 下拉刷新。

第一版可以不做复杂推荐算法。

店铺卡片

每个店铺展示：

- 店铺Logo；
- 店铺名称；
- 店铺公告；
- 营业状态；
- 营业时间；
- 起送金额；
- 配送费；
- 预计配送时长；
- 支持配送的小区。

店铺状态

店铺状态包括：

状态	说明	是否允许下单
营业中	正常营业	是
休息中	不在营业时间	否
暂停接单	临时停止接单	否
已停用	平台关闭店铺	否

休息中和暂停接单的店铺仍然可以展示，但需要显示状态并禁用结算。

5.4 店铺详情

店铺基本信息

展示：

- 店铺名称；
- Logo；
- 封面图；
- 店铺公告；
- 联系电话；
- 营业时间；
- 起送金额；
- 配送费；
- 预计配送时间；
- 支持配送小区；

- 当前营业状态。

商品区域

采用左侧分类、右侧商品列表布局。

店铺信息
店铺公告
商品区域
├── 左侧商品分类
└── 右侧商品列表
底部购物车栏

商品分类

分类由后台按店铺维护，例如：

- 热销；
- 早餐；
- 主食；
- 饮品；
- 零食；
- 日用品。

商品列表信息

- 商品图片；
- 商品名称；
- 商品描述；
- 当前价格；
- 原价；
- 已售数量；
- 剩余库存；
- 商品状态；
- 加减数量按钮。

商品状态

状态	前端表现
销售中	可以加入购物车
已售罄	显示售罄，不能购买
已下架	小程序不显示

5.5 商品详情

商品详情展示：

- 商品主图；
- 商品轮播图；
- 商品名称；
- 售价；
- 原价；
- 销量；
- 库存；
- 商品描述；
- 商品详情；
- 售后说明；
- 购买数量；
- 加入购物车按钮。

第一版商品详情不提供复杂规格选择。

5.6 购物车

数据设计

第一版建议使用服务端购物车，并同时在Pinia中保存当前页面状态。

服务端购物车可以实现：

- 跨页面同步；
- 换手机后恢复；
- 重新登录后恢复；
- 服务端统一校验店铺和商品。

购物车功能

- 查看商品；
- 增加数量；
- 减少数量；
- 删除单个商品；
- 清空购物车；
- 计算商品总额；
- 显示配送费；
- 判断是否达到起送金额；
- 进入订单确认页。

数量规则

- 商品数量不能小于1；
- 商品数量不能超过库存；
- 商品数量不能超过商品限购数量；
- 后台修改库存后，结算时需要重新校验。

金额计算

商品小计 = 商品单价 × 商品数量

商品总金额 = 所有商品小计之和

实付金额 = 商品总金额 + 配送费 - 优惠金额

第一版优惠金额固定为0，因此：

实付金额 = 商品总金额 + 配送费

起送金额

未达到起送金额时显示：

还差 ¥X 起送

达到起送金额后显示：

去结算

5.7 收货地址

地址字段

- 收货人姓名；
- 联系手机号；
- 所在小区；
- 楼栋；
- 单元；
- 门牌号；
- 补充地址；
- 地址标签；
- 是否默认地址。

地址标签

- 家；
- 公司；
- 学校；
- 其他。

地址规则

- 所在小区必须从平台配置的小区中选择；
- 用户不能手动创建不存在的小区；

- 用户可以保存多个地址；
- 同一用户只能有一个默认地址；
- 删除默认地址后，可以自动将最近使用地址设置为默认地址；
- 历史订单不受地址修改影响。

配送范围校验

下单时必须校验：

收货地址所属小区
是否在当前店铺配送范围内

不支持配送时提示：

当前店铺暂不支持配送到该小区，请更换地址或选择其他店铺。

5.8 订单确认

页面内容

- 店铺信息；
- 收货地址；
- 商品列表；
- 商品数量；
- 商品金额；
- 配送方式；
- 配送时间；
- 配送费；
- 用户备注；
- 实付金额；
- 提交订单按钮。

配送时间模式

尽快送达

商家接单后尽快准备和配送。

预约配送

用户从店铺启用的配送时间段中选择，例如：

- 07:00；
- 08:00；
- 11:00；
- 12:00；
- 18:00；
- 19:00。

配送时间校验

提交订单时，服务端需要校验：

- 时间段是否启用；
- 是否已超过停止下单时间；
- 是否达到最大订单数；
- 店铺当天是否营业；
- 预约日期是否允许下单。

订单备注

用户可以填写：

- 不要辣；
- 放门口；
- 到了打电话；
- 不要敲门；
- 其他要求。

备注长度建议限制在200字以内。

5.9 创建订单

创建订单前校验

Koa服务端必须校验：

1. 用户是否登录；
2. 用户是否绑定手机号；
3. 地址是否属于当前用户；
4. 地址小区是否启用；
5. 店铺是否存在；
6. 店铺是否营业；
7. 店铺是否暂停接单；
8. 店铺是否支持当前小区；
9. 商品是否属于当前店铺；
10. 商品是否上架；
11. 商品库存是否充足；
12. 商品购买数量是否超限；
13. 商品金额是否达到起送金额；
14. 配送时间是否有效；
15. 配送时间段是否已约满；
16. 请求是否重复提交。

金额安全

小程序端只提交：

- 商品ID；

- 购买数量；
- 地址ID；
- 配送时间；
- 用户备注。

小程序端传来的价格和总金额不能作为可信数据。

所有金额必须由Koa服务端重新查询商品并计算。

地址快照

创建订单时保存：

- 收货人姓名；
- 手机号；
- 小区名称；
- 楼栋；
- 单元；
- 门牌号；
- 完整地址。

用户之后修改地址，不能影响历史订单。

商品快照

创建订单时保存：

- 商品ID；
- 商品名称；
- 商品图片；
- 下单价格；
- 商品数量；
- 商品小计。

后台之后修改商品名称和价格，不能影响历史订单。

5.10 微信支付

支付流程

用户提交订单

- Koa创建待支付订单
- Koa调用微信支付下单接口
- 返回小程序支付参数
- `uni.requestPayment`调起微信支付
- 用户完成支付
- 微信支付服务器通知Koa
- Koa验证签名和支付金额
- 更新支付记录

- 更新订单为已支付待接单
- 写入订单状态日志

支付结果原则

小程序的支付成功回调只能用于页面提示。

订单是否真正支付成功，必须以：

微信支付服务端通知

为准。

支付状态

- 未支付；
- 支付处理中；
- 支付成功；
- 支付失败；
- 已关闭；
- 已退款；
- 部分退款。

支付超时

待支付订单15分钟内未支付：

- 自动关闭订单；
- 释放锁定库存；
- 更新订单状态为已取消；
- 记录系统操作日志。

5.11 订单列表

订单分类

- 全部；
- 待付款；
- 待接单；
- 已接单；
- 制作中；
- 待配送；
- 配送中；
- 已完成；
- 已取消；
- 退款中；
- 已退款。

订单卡片

展示：

- 店铺名称；
- 店铺Logo；
- 订单编号；
- 商品图片；
- 商品名称；
- 商品数量；
- 商品种类数量；
- 实付金额；
- 订单状态；
- 下单时间；
- 当前可执行操作。

用户操作

根据订单状态显示：

订单状态	用户操作
待付款	去支付、取消订单
待接单	查看详情、申请取消
已接单	查看详情、联系店铺
制作中	查看进度、联系店铺
待配送	查看进度
配送中	查看配送、联系店铺
已完成	再来一单
已取消	再来一单
已退款	再来一单

5.12 订单详情

展示：

- 订单编号；
- 当前状态；
- 状态说明；
- 配送进度；
- 店铺信息；
- 店铺联系电话；
- 商品明细；
- 收货地址快照；

- 配送方式；
- 配送时间；
- 用户备注；
- 商品金额；
- 配送费；
- 优惠金额；
- 实付金额；
- 支付时间；
- 接单时间；
- 制作时间；
- 开始配送时间；
- 完成时间；
- 取消时间；
- 取消原因；
- 退款信息。

5.13 配送进度

使用Vant Steps展示订单进度。

推荐节点：

1. 订单已支付；
2. 商家已接单；
3. 商品准备中；
4. 等待配送；
5. 配送中；
6. 已完成。

每个节点展示：

- 状态名称；
- 状态说明；
- 状态更新时间。

第一版不展示：

- 实时地图；
- 配送轨迹；
- 配送员实时位置。

5.14 订单取消

用户可以直接取消

- 待支付订单。

用户可以申请取消

- 已支付但商家未接单。

用户不能直接取消

- 商家已接单；
- 制作中；
- 待配送；
- 配送中；
- 已完成。

这些状态需要联系店铺或平台处理。

取消原因

- 不想要了；
 - 商品选错；
 - 地址填写错误；
 - 配送时间不合适；
 - 重复下单；
 - 店铺等待时间过长；
 - 其他。
-

5.15 退款

退款状态

- 待审核；
- 审核通过；
- 审核拒绝；
- 退款处理中；
- 退款成功；
- 退款失败。

退款信息

- 退款编号；
- 原订单编号；
- 退款金额；
- 退款原因；
- 用户说明；
- 管理员处理说明；
- 申请时间；
- 审核时间；
- 完成时间。

第一版处理方式

第一版退款由平台管理员在后台审核。

审核通过后，由Koa服务端调用微信退款接口。

5.16 个人中心

包含：

- 用户头像；
- 用户昵称；
- 手机号；
- 我的订单；
- 收货地址；
- 联系客服；
- 用户协议；
- 隐私政策；
- 关于平台；
- 退出登录。

第一版暂不开发：

- 收藏店铺；
 - 优惠券；
 - 会员等级；
 - 积分；
 - 浏览记录。
-

六、小程序页面清单

6.1 TabBar页面

1. 首页；
2. 订单；
3. 我的。

6.2 普通页面

1. 小区选择页；
2. 店铺搜索页；
3. 店铺详情页；
4. 商品详情页；
5. 购物车弹层或购物车页；
6. 订单确认页；
7. 支付结果页；
8. 订单详情页；
9. 配送进度页；
10. 地址列表页；
11. 新增地址页；
12. 编辑地址页；

- 13. 退款申请页；
 - 14. 退款详情页；
 - 15. 用户协议页；
 - 16. 隐私政策页；
 - 17. 联系客服页。
-

七、后台管理系统需求

7.1 后台登录

功能

- 账号密码登录；
- 登录验证码；
- JWT登录状态；
- 退出登录；
- 修改密码；
- 登录失败限制；
- 登录日志。

密码安全

管理员密码必须使用bcrypt哈希保存。

禁止保存明文密码。

7.2 数据看板

核心指标

- 店铺总数；
- 营业中店铺数；
- 用户总数；
- 今日新增用户；
- 今日订单数；
- 今日支付订单数；
- 今日成交金额；
- 待接单订单数；
- 制作中订单数；
- 配送中订单数；
- 退款中订单数。

店铺数据

按店铺展示：

- 今日订单量；
- 今日成交金额；

- 待处理订单；
- 配送中订单；
- 商品数量；
- 库存预警数量。

图表

可以使用ECharts展示：

- 最近7天订单趋势；
 - 最近7天成交金额；
 - 店铺订单排行；
 - 热销商品排行；
 - 小区订单排行。
-

7.3 店铺管理

店铺字段

- 店铺ID；
- 店铺名称；
- 店铺Logo；
- 店铺封面；
- 店铺简介；
- 店铺公告；
- 联系电话；
- 店铺地址；
- 营业开始时间；
- 营业结束时间；
- 起送金额；
- 默认配送费；
- 预计配送时长；
- 店铺状态；
- 排序值；
- 创建时间；
- 更新时间。

店铺操作

- 新增；
- 编辑；
- 查看详情；
- 启用；
- 停用；
- 暂停接单；
- 恢复接单；
- 配置配送小区；
- 配置配送时间；
- 查看商品；
- 查看订单。

删除规则

已经产生商品或订单的店铺不能物理删除。

可以将店铺设置为停用。

7.4 商品分类管理

分类字段

- 分类ID；
- 分类名称；
- 所属店铺；
- 排序值；
- 状态；
- 创建时间。

分类操作

- 新增；
- 编辑；
- 启用；
- 停用；
- 调整排序；
- 删除未使用分类。

已绑定商品的分类不能直接删除。

7.5 商品管理

商品字段

- 商品ID；
- 商品名称；
- 所属店铺；
- 商品分类；
- 商品主图；
- 商品轮播图；
- 商品简介；
- 商品详情；
- 售价；
- 原价；
- 库存；
- 销量；
- 限购数量；
- 是否热销；
- 商品状态；
- 排序值；
- 创建时间；

- 更新时间。

商品操作

- 新增商品；
- 编辑商品；
- 上架；
- 下架；
- 设置售罄；
- 修改库存；
- 复制商品；
- 查看销量；
- 软删除商品。

库存预警

后台可以设置库存预警值。

当库存低于预警值时，在商品列表显示提示。

7.6 配送小区管理

小区字段

- 小区ID；
- 小区名称；
- 城市；
- 区县；
- 详细地址；
- 状态；
- 排序值；
- 创建时间。

小区操作

- 新增小区；
- 编辑小区；
- 启用；
- 停用；
- 调整排序。

小区停用规则

小区停用后：

- 小程序不再展示；
 - 用户不能选择该小区创建新地址；
 - 已有历史地址和订单继续保留；
 - 已有关联店铺不能继续接收该小区新订单。
-

7.7 店铺配送范围管理

每个店铺可以关联多个小区。

配置字段

- 店铺；
- 小区；
- 是否支持配送；
- 起送金额；
- 配送费；
- 预计配送时长；
- 是否暂停该小区配送。

配置优先级

店铺可以配置默认：

- 起送金额；
- 配送费；
- 配送时长。

某个小区可以设置独立值。

计算时优先使用：

店铺小区独立配置

没有独立配置时，使用：

店铺默认配置

7.8 配送时间管理

配置字段

- 所属店铺；
- 配送时间；
- 停止下单时间；
- 最大订单数；
- 是否启用；
- 排序值。

示例

配送时间	停止下单时间	最大订单数
07:00	06:30	20
08:00	07:30	20
11:00	10:30	30
12:00	11:30	30
18:00	17:30	30
19:00	18:30	30

达到最大订单数后，小程序显示：

当前配送时间已约满

7.9 订单管理

订单筛选

支持：

- 订单编号；
- 用户昵称；
- 用户手机号；
- 店铺；
- 小区；
- 订单状态；
- 支付状态；
- 配送方式；
- 配送日期；
- 配送时间；
- 下单时间；
- 支付时间。

订单列表字段

- 订单编号；
- 用户；
- 手机号；
- 店铺；
- 小区；
- 商品数量；
- 商品金额；
- 配送费；
- 实付金额；
- 支付状态；
- 订单状态；

- 预约配送时间；
- 下单时间；
- 操作。

订单详情

展示：

- 用户信息；
 - 店铺信息；
 - 商品快照；
 - 收货地址快照；
 - 金额明细；
 - 支付信息；
 - 配送信息；
 - 用户备注；
 - 后台备注；
 - 状态变更记录；
 - 退款记录；
 - 操作记录。
-

7.10 订单状态操作

待付款

后台可以：

- 查看详情；
- 关闭订单。

已支付待接单

后台可以：

- 接单；
- 拒单；
- 取消订单；
- 查看详情。

已接单

后台可以：

- 开始制作；
- 取消订单；
- 修改后台备注。

制作中

后台可以：

- 标记制作完成；
- 修改后台备注。

待配送

后台可以：

- 开始配送；
- 修改配送时间。

配送中

后台可以：

- 标记已送达；
- 联系用户。

已完成

后台只能查看，不允许再次修改正常业务状态。

所有订单状态修改都必须通过专用接口和状态机校验，不能使用一个任意修改状态的通用接口。

7.11 用户管理

用户字段

- 用户ID；
- 微信昵称；
- 头像；
- 手机号；
- 当前小区；
- 注册时间；
- 最近登录时间；
- 有效订单数量；
- 累计消费金额；
- 用户状态。

用户操作

- 查看用户详情；
- 查看用户订单；
- 禁用用户；
- 恢复用户；
- 添加后台备注。

管理员不能查看用户完整OpenID等敏感字段，除非确有业务需要。

7.12 退款管理

退款列表字段

- 退款编号；
- 订单编号；
- 用户；
- 店铺；
- 退款金额；
- 退款原因；
- 退款状态；
- 申请时间；
- 处理时间。

操作

- 查看退款详情；
- 同意退款；
- 拒绝退款；
- 调用微信退款；
- 查询微信退款状态；
- 添加处理备注。

7.13 管理员和权限

第一版

第一版只实现超级管理员。

超级管理员拥有全部后台权限。

第二阶段

增加：

- 平台运营；
- 客服；
- 店铺管理员。

权限可以细分为：

- 查看；
- 新增；
- 编辑；
- 删除；
- 状态修改；
- 退款；
- 导出。

7.14 操作日志

以下操作必须记录日志：

- 登录后台；
- 新增或编辑店铺；
- 修改店铺状态；
- 修改配送范围；
- 新增或编辑商品；
- 修改商品价格；
- 修改商品库存；
- 修改订单状态；
- 取消订单；
- 处理退款；
- 禁用用户；
- 修改管理员。

日志字段：

- 操作人ID；
- 操作人名称；
- 操作模块；
- 操作类型；
- 业务数据ID；
- 操作说明；
- 修改前数据；
- 修改后数据；
- 请求IP；
- 请求路径；
- 操作时间。

八、后台页面清单

1. 后台登录页；
2. 数据看板；
3. 店铺列表页；
4. 新增店铺页；
5. 编辑店铺页；
6. 店铺详情页；
7. 店铺配送小区配置页；
8. 店铺配送时间配置页；
9. 商品分类列表页；
10. 商品列表页；
11. 新增商品页；
12. 编辑商品页；
13. 小区列表页；
14. 订单列表页；
15. 订单详情页；

- 16. 退款列表页；
- 17. 退款详情页；
- 18. 用户列表页；
- 19. 用户详情页；
- 20. 管理员列表页；
- 21. 操作日志页；
- 22. 系统配置页。

九、订单状态设计

9.1 订单状态

状态值	中文名称	说明
pending_payment	待付款	已创建订单，尚未付款
paid	已支付待接单	微信支付成功
accepted	已接单	店铺已确认订单
preparing	制作中	店铺正在准备商品
waiting_delivery	待配送	商品准备完成
delivering	配送中	商品正在配送
completed	已完成	商品已经送达
cancelled	已取消	订单已经取消
refund_pending	退款中	正在处理退款
refunded	已退款	退款已经完成

9.2 正常状态流转

```
pending_payment
→ paid
→ accepted
→ preparing
→ waiting_delivery
→ delivering
→ completed
```

9.3 取消分支

```
pending_payment
→ cancelled
```


paid
→ refund_pending
→ refunded

accepted
→ refund_pending
→ refunded

9.4 状态机规则

当前状态	可以进入的状态
pending_payment	paid、cancelled
paid	accepted、refund_pending
accepted	preparing、refund_pending
preparing	waiting_delivery
waiting_delivery	delivering
delivering	completed
refund_pending	refunded
completed	不允许修改
cancelled	不允许修改
refunded	不允许修改

9.5 状态日志

每次状态变化记录：

- 订单ID；
- 原状态；
- 新状态；
- 操作人类型；
- 操作人ID；
- 操作人名称；
- 操作说明；
- 操作时间。

操作人类型包括：

- user；
- admin；
- system；
- wechat。

十、库存处理方案

10.1 推荐方案

第一版采用：

创建待支付订单时锁定并扣减可售库存

创建订单时

- 校验库存；
- 使用数据库事务扣减库存；
- 创建订单；
- 创建订单商品快照；
- 写入订单状态日志。

支付成功时

- 更新订单为已支付；
- 更新支付记录；
- 增加商品销量；
- 不再重复扣减库存。

支付超时或取消时

- 恢复库存；
- 更新订单为已取消；
- 标记库存已经释放。

10.2 防止重复恢复库存

订单表增加：

stock_released

含义：

- 0：尚未释放；
- 1：已经释放。

释放库存时必须先判断该字段，避免定时任务和用户取消操作重复恢复库存。

10.3 并发处理

扣减库存必须使用带条件的数据库更新，例如：

```
UPDATE products
SET stock = stock - ?
WHERE id = ?
AND stock >= ?;
```

根据受影响行数判断库存是否充足。

十一、支付和退款规则

11.1 支付安全

- 微信支付回调必须验签；
- 回调订单号必须存在；
- 回调金额必须与订单金额一致；
- 相同支付回调必须支持重复通知；
- 重复通知不能重复增加销量；
- 已支付订单不能重复创建支付记录。

11.2 退款安全

- 退款金额不能超过实际支付金额；
 - 同一订单累计退款不能超过实付金额；
 - 微信退款结果需要保存；
 - 退款接口必须防止重复提交；
 - 退款成功后更新订单状态；
 - 退款失败需要保留失败原因。
-

十二、关键业务规则

12.1 单店铺订单

一个订单只能购买一家店铺的商品。

12.2 小区配送校验

收货地址所属小区必须在店铺配送范围内。

12.3 营业状态校验

店铺处于以下状态时不能提交订单：

- 休息中；
- 暂停接单；
- 已停用。

12.4 服务端金额计算

订单金额只能由Koa服务端计算。

前端传入的价格不能作为订单依据。

12.5 订单快照

历史订单必须保存商品和地址快照。

12.6 防重复提交

创建订单请求必须包含唯一请求ID。

同一用户不能使用同一个请求ID创建多个订单。

12.7 订单超时

待支付订单超过15分钟自动关闭并恢复库存。

12.8 店铺接单提醒

订单支付后超过10分钟未接单：

- 后台显示超时提醒；
- 播放新订单或超时提示音；
- 第一版不自动退款。

12.9 数据删除

以下数据采用软删除：

- 店铺；
- 商品；
- 商品分类；
- 小区；
- 地址。

以下数据禁止删除：

- 订单；
- 订单商品；
- 支付记录；
- 退款记录；
- 状态日志；
- 操作日志。

十三、消息通知

13.1 用户通知

通过微信订阅消息通知：

- 支付成功；
- 店铺已接单；
- 商品开始配送；
- 商品已送达；
- 订单取消；
- 退款成功。

13.2 后台通知

后台管理系统需要提供：

- 新订单声音提醒；
- 待接单数量角标；
- 超时未接单提醒；
- 退款申请提醒；
- 库存不足提醒。

十四、非功能需求

14.1 安全

- 用户接口使用JWT鉴权；
- 后台接口使用独立管理员JWT；
- 用户Token和管理员Token不能混用；
- 接口校验数据归属关系；
- 管理员密码使用bcrypt；
- 微信支付回调验签；
- 关键接口使用幂等机制；
- 用户手机号后台脱敏；
- 上传文件限制类型和大小；
- 防止SQL注入；
- 配置接口访问频率限制；
- 记录关键后台操作。

14.2 性能

第一版目标：

- 普通查询接口响应时间尽量小于1秒；
- 店铺、商品、订单列表使用分页；
- 店铺和商品查询建立索引；

- 首页数据可以使用Redis缓存；
- 图片使用对象存储和CDN；
- 避免一次接口查询过多关联数据。

14.3 数据一致性

以下操作必须使用数据库事务：

- 创建订单；
- 扣减库存；
- 创建订单商品；
- 创建订单状态日志；
- 支付成功处理；
- 取消订单并恢复库存；
- 退款成功处理。

14.4 日志

服务端至少保留：

- 请求日志；
- 错误日志；
- 支付回调日志；
- 退款回调日志；
- 定时任务日志；
- 后台操作日志。

日志中不能直接输出：

- 用户Token；
- 完整支付密钥；
- 完整手机号；
- 微信支付敏感数据。

十五、核心数据表

第一版建议包含：

1. `users`：用户表；
2. `user_addresses`：用户地址表；
3. `admins`：管理员表；
4. `roles`：角色表；
5. `admin_roles`：管理员角色关联表；
6. `stores`：店铺表；
7. `communities`：小区表；
8. `store_communities`：店铺配送小区关联表；
9. `product_categories`：商品分类表；
10. `products`：商品表；
11. `shopping_cart_items`：购物车商品表；

12. `delivery_time_slots` : 配送时间段表;
13. `orders` : 订单表;
14. `order_items` : 订单商品快照表;
15. `order_status_logs` : 订单状态日志表;
16. `payments` : 支付记录表;
17. `refunds` : 退款记录表;
18. `operation_logs` : 后台操作日志表;
19. `notices` : 平台公告表;
20. `system_configs` : 系统配置表。

第一版暂时不建立复杂SKU表。

后续增加商品规格时,再新增:

- `product_skus` ;
- `product_spec_groups` ;
- `product_spec_values` 。

十六、接口模块规划

16.1 小程序接口

统一前缀:

/api/v1

登录与用户

`POST /api/v1/auth/wechat-login`
`POST /api/v1/users/bind-phone`
`GET /api/v1/users/profile`
`PUT /api/v1/users/current-community`

小区

`GET /api/v1/communities`

店铺

`GET /api/v1/stores`
`GET /api/v1/stores/:storeId`
`GET /api/v1/stores/:storeId/products`
`GET /api/v1/stores/:storeId/delivery-slots`

商品

GET /api/v1/products/:productId

购物车

GET /api/v1/cart

POST /api/v1/cart/items

PUT /api/v1/cart/items/:itemId

DELETE /api/v1/cart/items/:itemId

DELETE /api/v1/cart

地址

GET /api/v1/addresses

POST /api/v1/addresses

PUT /api/v1/addresses/:addressId

DELETE /api/v1/addresses/:addressId

PUT /api/v1/addresses/:addressId/default

订单

POST /api/v1/orders/preview

POST /api/v1/orders

GET /api/v1/orders

GET /api/v1/orders/:orderId

POST /api/v1/orders/:orderId/cancel

POST /api/v1/orders/:orderId/confirm-receipt

支付

POST /api/v1/payments/wechat

POST /api/v1/payments/wechat/notify

GET /api/v1/payments/orders/:orderId/status

退款

POST /api/v1/orders/:orderId/refunds

GET /api/v1/refunds/:refundId

16.2 后台接口

统一前缀：

/api/v1/admin

登录

POST /api/v1/admin/auth/login
GET /api/v1/admin/auth/profile
POST /api/v1/admin/auth/logout
PUT /api/v1/admin/auth/password

数据看板

GET /api/v1/admin/dashboard/summary
GET /api/v1/admin/dashboard/order-trend
GET /api/v1/admin/dashboard/store-ranking
GET /api/v1/admin/dashboard/recent-orders

店铺

GET /api/v1/admin/stores
POST /api/v1/admin/stores
GET /api/v1/admin/stores/:storeId
PUT /api/v1/admin/stores/:storeId
PUT /api/v1/admin/stores/:storeId/status

店铺配送范围

GET /api/v1/admin/stores/:storeId/communities
PUT /api/v1/admin/stores/:storeId/communities

配送时间

GET /api/v1/admin/stores/:storeId/delivery-slots
POST /api/v1/admin/stores/:storeId/delivery-slots
PUT /api/v1/admin/delivery-slots/:slotId
DELETE /api/v1/admin/delivery-slots/:slotId

商品分类

GET /api/v1/admin/categories
POST /api/v1/admin/categories
PUT /api/v1/admin/categories/:categoryId
DELETE /api/v1/admin/categories/:categoryId

商品

```
GET /api/v1/admin/products
POST /api/v1/admin/products
GET /api/v1/admin/products/:productId
PUT /api/v1/admin/products/:productId
PUT /api/v1/admin/products/:productId/status
PUT /api/v1/admin/products/:productId/stock
```

小区

```
GET /api/v1/admin/communities
POST /api/v1/admin/communities
PUT /api/v1/admin/communities/:communityId
PUT /api/v1/admin/communities/:communityId/status
```

订单

```
GET /api/v1/admin/orders
GET /api/v1/admin/orders/:orderId

POST /api/v1/admin/orders/:orderId/accept
POST /api/v1/admin/orders/:orderId/reject
POST /api/v1/admin/orders/:orderId/start-preparing
POST /api/v1/admin/orders/:orderId/ready-for-delivery
POST /api/v1/admin/orders/:orderId/start-delivery
POST /api/v1/admin/orders/:orderId/complete
POST /api/v1/admin/orders/:orderId/cancel
PUT /api/v1/admin/orders/:orderId/remark
```

用户

```
GET /api/v1/admin/users
GET /api/v1/admin/users/:userId
GET /api/v1/admin/users/:userId/orders
PUT /api/v1/admin/users/:userId/status
```

退款

```
GET /api/v1/admin/refunds
GET /api/v1/admin/refunds/:refundId
POST /api/v1/admin/refunds/:refundId/approve
POST /api/v1/admin/refunds/:refundId/reject
```

十七、Koa服务端项目结构

```
server/
├── src/
│   ├── app.ts
│   ├── server.ts
│   ├── config/
│   │   ├── env.ts
│   │   ├── database.ts
│   │   ├── redis.ts
│   │   └── wechat.ts
│   ├── routes/
│   │   ├── index.ts
│   │   ├── miniapp/
│   │   └── admin/
│   ├── controllers/
│   │   ├── auth.controller.ts
│   │   ├── store.controller.ts
│   │   ├── product.controller.ts
│   │   ├── cart.controller.ts
│   │   ├── address.controller.ts
│   │   ├── order.controller.ts
│   │   ├── payment.controller.ts
│   │   └── admin/
│   ├── services/
│   │   ├── auth.service.ts
│   │   ├── store.service.ts
│   │   ├── product.service.ts
│   │   ├── cart.service.ts
│   │   ├── address.service.ts
│   │   ├── order.service.ts
│   │   ├── payment.service.ts
│   │   └── refund.service.ts
│   ├── repositories/
│   ├── middlewares/
│   │   ├── auth.middleware.ts
│   │   ├── admin-auth.middleware.ts
│   │   ├── error.middleware.ts
│   │   ├── logger.middleware.ts
│   │   ├── validator.middleware.ts
│   │   └── rate-limit.middleware.ts
│   ├── validators/
│   ├── enums/
│   ├── constants/
│   ├── utils/
│   ├── jobs/
│   │   ├── close-expired-orders.job.ts
│   │   └── check-payment-status.job.ts
│   ├── types/
│   └── prisma/
```

```
├── prisma/
│   ├── schema.prisma
│   └── migrations/
│       └── seed.ts
├── tests/
├── uploads/
├── Dockerfile
├── package.json
└── tsconfig.json
```

分层职责

- Router：定义接口路径；
- Controller：接收参数并返回结果；
- Service：处理业务逻辑；
- Repository：封装数据库操作；
- Middleware：鉴权、错误处理、日志和校验；
- Job：定时任务；
- Validator：请求参数校验。

Controller中不要直接编写复杂SQL和核心业务逻辑。

十八、小程序项目结构

```
miniapp/
├── src/
│   ├── pages/
│   │   ├── home/
│   │   ├── order/
│   │   ├── user/
│   │   ├── community/
│   │   ├── store/
│   │   ├── product/
│   │   ├── address/
│   │   └── refund/
│   ├── components/
│   │   ├── StoreCard/
│   │   ├── ProductCard/
│   │   ├── CartBar/
│   │   ├── OrderCard/
│   │   └── EmptyState/
│   ├── stores/
│   │   ├── user.ts
│   │   ├── community.ts
│   │   ├── cart.ts
│   │   └── order.ts
│   ├── api/
│   └── auth.ts
```

```

|   |   |—— community.ts
|   |   |—— store.ts
|   |   |—— product.ts
|   |   |—— cart.ts
|   |   |—— address.ts
|   |   |—— order.ts
|   |   |—— payment.ts
|   |—— utils/
|   |   |—— request.ts
|   |   |—— auth.ts
|   |   |—— storage.ts
|   |   |—— money.ts
|   |—— types/
|   |—— static/
|   |—— App.vue
|   |—— main.ts
|   |—— pages.json
|   |—— manifest.json
|—— package.json
|—— vite.config.ts

```

十九、后台项目结构

```

admin-web/
|—— src/
|   |—— api/
|   |—— assets/
|   |—— components/
|   |—— layouts/
|   |—— router/
|   |—— stores/
|   |—— utils/
|   |—— types/
|   |—— views/
|       |—— login/
|       |—— dashboard/
|       |—— stores/
|       |—— categories/
|       |—— products/
|       |—— communities/
|       |—— orders/
|       |—— refunds/
|       |—— users/
|       |—— system/
|   |—— App.vue
|   |—— main.ts

```

```
└── package.json
└── vite.config.ts
```

二十、统一接口响应格式

成功响应

```
{
  "code": 0,
  "message": "success",
  "data": {}
}
```

分页响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "list": [],
    "page": 1,
    "pageSize": 20,
    "total": 100,
    "totalPages": 5
  }
}
```

失败响应

```
{
  "code": "PRODUCT_STOCK_NOT_ENOUGH",
  "message": "商品库存不足",
  "data": null
}
```

常见业务错误码

- UNAUTHORIZED ;
- FORBIDDEN ;
- USER_DISABLED ;
- PHONE_NOT_BOUND ;
- COMMUNITY_NOT_FOUND ;
- STORE_NOT_FOUND ;

- STORE_NOT_DELIVERABLE ;
 - STORE_CLOSED ;
 - STORE_PAUSED ;
 - PRODUCT_NOT_FOUND ;
 - PRODUCT_OFF_SHELF ;
 - PRODUCT_STOCK_NOT_ENOUGH ;
 - MINIMUM_ORDER_NOT_REACHED ;
 - DELIVERY_SLOT_UNAVAILABLE ;
 - ADDRESS_NOT_FOUND ;
 - ORDER_NOT_FOUND ;
 - INVALID_ORDER_STATUS ;
 - ORDER_ALREADY_PAID ;
 - PAYMENT_FAILED ;
 - DUPLICATE_REQUEST 。
-

二十一、第一版MVP范围

21.1 小程序MVP

必须完成：

- 微信登录；
- 小区选择；
- 店铺列表；
- 店铺搜索；
- 店铺详情；
- 商品分类；
- 商品列表；
- 商品详情；
- 购物车；
- 收货地址；
- 订单预览；
- 创建订单；
- 微信支付；
- 订单列表；
- 订单详情；
- 状态式配送进度；
- 取消未支付订单；
- 联系店铺。

21.2 后台MVP

必须完成：

- 管理员登录；
- 数据看板基础指标；
- 店铺管理；
- 商品分类管理；

- 商品管理；
- 小区管理；
- 店铺配送范围；
- 配送时间配置；
- 订单列表；
- 订单详情；
- 订单状态修改；
- 用户列表；
- 基础退款处理；
- 操作日志。

21.3 服务端MVP

必须完成：

- 微信登录；
- JWT鉴权；
- 管理员登录；
- 店铺和商品接口；
- 购物车接口；
- 地址接口；
- 订单事务；
- 库存锁定和恢复；
- 微信支付；
- 支付回调；
- 超时订单关闭；
- 订单状态机；
- 后台统计接口；
- 统一异常处理；
- 请求日志。

21.4 第一版暂不开发

- 商家自主入驻；
- 商家独立后台；
- 配送员端；
- 实时地图定位；
- 配送轨迹；
- 复杂SKU；
- 优惠券；
- 满减；
- 秒杀；
- 拼团；
- 会员积分；
- 商品评价；
- 平台抽佣；
- 商家结算；
- 多店铺合并支付；
- 发票；
- 售后工单系统。

二十二、开发阶段

第一阶段：基础工程

完成：

- 创建Git仓库；
- 创建uni-app项目；
- 接入Vant Weapp；
- 创建Vue 3后台项目；
- 接入Element Plus；
- 创建Koa TypeScript项目；
- Docker启动MySQL和Redis；
- 初始化Prisma；
- 配置ESLint和Prettier；
- 配置环境变量；
- 完成健康检查接口。

第二阶段：后台基础数据

开发顺序：

1. 后台登录；
2. 小区管理；
3. 店铺管理；
4. 店铺配送范围；
5. 商品分类；
6. 商品管理；
7. 配送时间管理。

阶段成果：

管理员可以在后台创建完整的店铺和商品基础数据

第三阶段：小程序浏览链路

开发顺序：

1. 微信登录；
2. 小区选择；
3. 首页店铺列表；
4. 店铺搜索；
5. 店铺详情；
6. 商品分类；
7. 商品详情；
8. 购物车。

阶段成果：

用户可以浏览店铺和商品，并将商品加入购物车

第四阶段：地址和订单

开发顺序：

1. 地址管理；
2. 订单预览；
3. 创建订单；
4. 库存锁定；
5. 订单列表；
6. 订单详情；
7. 后台订单列表；
8. 订单状态流转。

阶段成果：

不接入真实支付，也能通过模拟支付跑通完整订单流程

第五阶段：微信支付

开发：

- 微信支付配置；
- 微信支付下单；
- 小程序调起支付；
- 支付回调；
- 支付结果查询；
- 订单超时关闭；
- 库存恢复；
- 支付幂等处理。

第六阶段：退款和通知

开发：

- 用户申请退款；
- 后台审核退款；
- 微信退款；
- 退款状态查询；
- 微信订阅消息；
- 后台新订单提醒；
- 库存预警。

第七阶段：测试和上线

完成：

- 接口测试；
- 订单并发测试；
- 库存超卖测试；
- 重复提交测试；
- 支付重复回调测试；
- 退款测试；
- 越权测试；
- 小程序隐私配置；
- 小程序审核；
- Docker部署；
- HTTPS；
- 正式上线。

二十三、第一阶段任务清单

建议首先完成以下任务：

1. 创建项目仓库；
2. 创建uni-app Vue 3项目；
3. 接入TypeScript和Pinia；
4. 接入Vant Weapp；
5. 创建Vue 3后台项目；
6. 接入Element Plus；
7. 创建Koa TypeScript服务端；
8. 配置Koa路由和统一异常处理中间件；
9. Docker启动MySQL和Redis；
10. 初始化Prisma；
11. 创建管理员表；
12. 创建小区表；
13. 创建店铺表；
14. 创建店铺配送小区关联表；
15. 创建商品分类表；
16. 创建商品表；
17. 完成后台管理员登录；
18. 完成小区管理；
19. 完成店铺管理；
20. 完成商品管理。

第一阶段完成后，系统应具备以下能力：

后台管理员登录

→ 创建小区

→ 创建店铺

→ 配置配送小区

- 创建商品分类
- 创建商品
- 小程序查询并展示店铺和商品

二十四、第一版默认产品决策

为了避免开发过程中反复修改，第一版默认采用：

- 多店铺模式；
- 平台统一管理店铺；
- 暂不开放商家入驻；
- 一个购物车只允许一家店铺；
- 一个订单只属于一家店铺；
- 商家自行配送；
- 用户手动选择固定小区；
- 不使用地图自动定位；
- 支持尽快送达和预约配送；
- 每家店铺独立配置配送小区；
- 每家店铺独立配置起送金额；
- 每家店铺独立配置配送费；
- 支持配送时间段订单上限；
- 商品采用单规格；
- 创建订单时锁定库存；
- 待支付订单15分钟自动关闭；
- 订单状态由后台管理员修改；
- 第一版不做优惠券和满减；
- 第一版不做实时配送地图；
- 第一版不做商家结算和平台抽佣；
- 支付成功以后才计入有效订单和成交金额。